

Agentic OS

A voice-first, model-agnostic "Jarvis" layer with a macOS-grade interface - and where Jev fits.

Working name: Nova (placeholder - see naming ideas at the end)

Status: Brainstorm, not a spec

Prepared for: Emmanuel Larbi

The one-line idea. You talk; it acts. A fast decision brain (Jev) listens continuously and decides *what* should happen in under half a second. A pluggable reasoning brain (Claude, Codex, Gemini, a local model - your choice) is called only when real thinking or text generation is needed. Everything is wrapped in a glassy, animated, macOS-inspired shell.

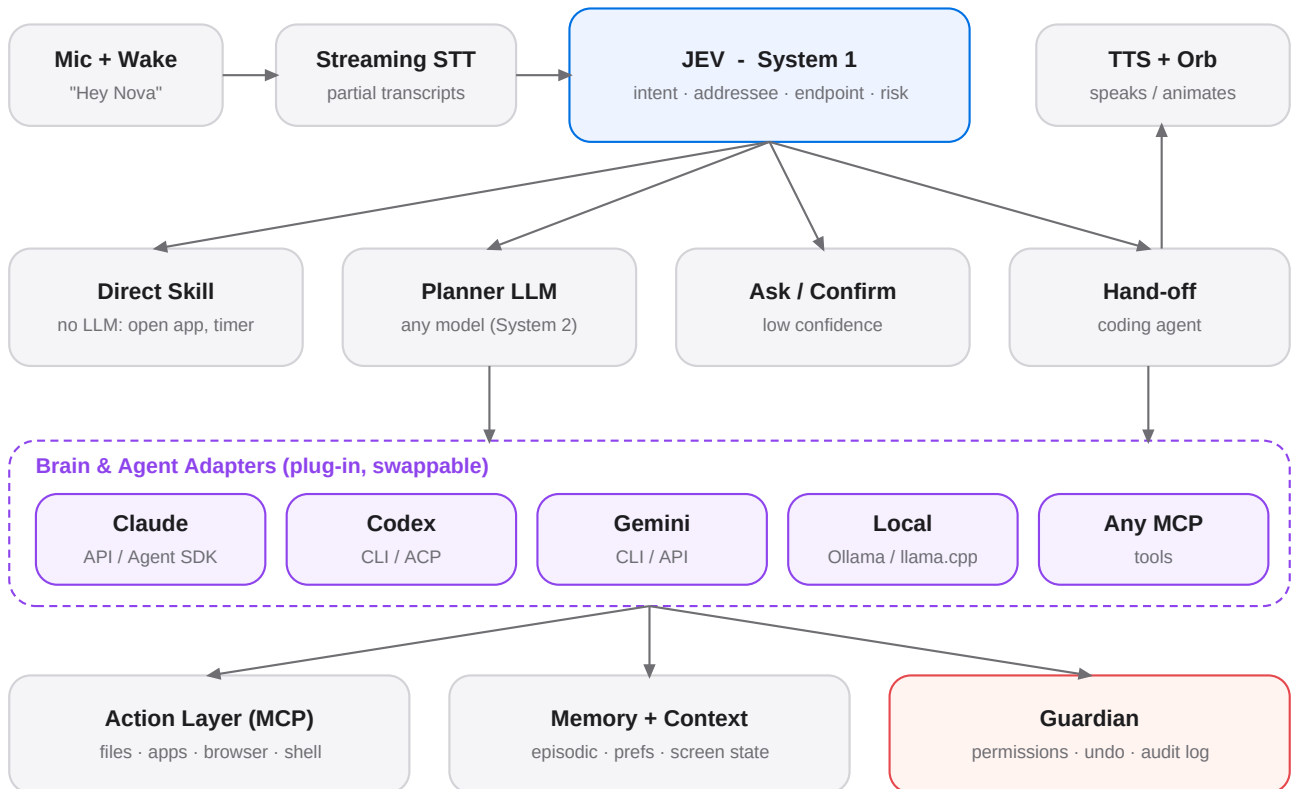
1. The key insight: two brains, not one

Most voice assistants feel slow because every utterance goes through a big LLM that thinks, then writes, then gets parsed. Jev changes the maths. It does not generate text; it returns typed decisions (pick-one, score, yes/no) with calibrated confidence in about 70-500 ms, at \$0.042 per million input tokens with free output. That makes it an ideal **System 1** for the OS - reflexes - while an LLM of your choice is **System 2** - deliberate work.

	System 1 - Jev	System 2 - pluggable LLM / agent
Job	Route, classify, gate, pick the next UI action	Plan multi-step tasks, write text/code, explain
Speed	~70-500 ms	seconds to minutes
Output	Typed values + probabilities; never off-schema	Free text / tool calls
Runs	On every utterance, often several times per second	Only when the router says so
Can't do	Generate text, arithmetic, dates, see audio/images	Be cheap or instant

This is also what makes the OS genuinely model-agnostic: the router, safety gates and UI never depend on which System 2 you plugged in.

2. Architecture at a glance



Flow: wake word → streaming speech-to-text → Jev decides intent, whether you were even talking to the OS, whether you've finished speaking, and how risky the action is → either a direct skill fires (no LLM at all), a planner LLM takes over, the OS asks you to confirm, or the task is handed to a coding agent. Every action passes through the Guardian before touching your machine.

3. Where Jev earns its place (concrete uses)

Each row is one Jev call with several questions answered in parallel - adding a question costs tokens but almost no time.

#	Decision	Jev question type	Why it matters for voice
1	Addressee detection: "Was that said to me?"	Noul (yes/no prob.)	Stops the OS reacting to TV, calls or side-conversations. Your own voice note to me contained stray background speech - exactly this problem.
2	Smart end-of-turn: "Is the thought finished?"	Noul on partial transcript	Lets it reply faster than silence-timeout endpointing without cutting you off mid-"uh".
3	Intent routing	Choice (up to 255 skills)	"Open Figma", "set a timer" never touch an LLM.
4	Complexity / which brain	Score + Choice	Simple → local model; hard → Claude/Codex. Cost control built in.
5	Risk tier of the action	Score (read-only → irreversible)	Drives whether to just act, confirm verbally, or require a tap.
6	Next UI action for computer use	Choice over indexed elements	Community demos drive browsers/Macs at ~\$0.0002 per step.
7	Interruption / barge-in intent	Choice: stop / correct / continue	"No, the other file" mid-reply gets handled correctly.

8	Verifier on LLM output	Noul / Score	Did the agent actually do what was asked? Catch before speaking.
9	Proactive nudges	Score (relevance / urgency)	Cheap enough to watch notifications and calendar continuously.

4. Jev's limits - design around them

- **Text in, decisions out.** No audio or images - always transcribe/OCR first. No text generation - spoken replies come from a template or System 2.
- **Reads literally.** Write criteria carefully; add an explicit *"other / none"* option to every Choice.
- **Not a calculator.** Counting, dates and maths live in ordinary code.
- **Context rot.** Send only the fields a question needs (64k total limit; smaller is more accurate).
- **Prompt injection is real.** Check Point flipped Jev's verdict for about 50 cents per break using a doctored document; typed input and "ignore instructions" warnings didn't help. Never let Jev alone authorise risky actions based on untrusted content (web pages, emails, files).
- **Early access.** Waitlisted; rate limits 1,200 req/min; pin the model version (e.g. jev-1.13.0) because answers can shift between releases. Benchmarks are self-reported.
- **Cloud-only.** Service is US West Coast based - expect extra latency from Ghana/Europe; measure it early.

Mitigation: put Jev behind an interface (DecisionEngine) with a fallback that asks any LLM for structured output (TypeSafe publishes a System One adapter for this). Then the OS still works offline or if Jev access is delayed.

5. Model-agnostic by design

Layer	Contract	Examples to plug in
Decision engine	decide(state, questions) → typed answers	Jev (default), LLM-with-schema fallback, local classifier
Reasoning brain	chat / plan with tool calls	Claude, GPT, Gemini, DeepSeek, Ollama models
Coding agents	Agent Client Protocol (ACP) or CLI wrapper	Claude Code, Codex CLI, Gemini CLI
Tools	MCP servers	Filesystem, browser, calendar, Slack, GitHub, home automation
Voice	STT / TTS / wake-word providers	Cloud streaming STT, on-device Whisper; neural TTS; openWakeWord

Users add a brain from a Settings sheet ("Add Claude", paste key, done). A per-task policy decides which brain handles what - e.g. code → Codex, writing → Claude, private files → local model only.

6. The interface: macOS-grade, voice-first

Inspired by macOS's Liquid Glass language - translucency, depth, rounded corners, soft motion - but with original icons, name and branding (a pixel clone of Apple's UI/assets invites trademark trouble and is less memorable anyway).

Signature components

- **The Orb.** A glass sphere at the centre-bottom that breathes when idle, ripples with your voice amplitude, swirls while thinking and pulses as it speaks. The emotional core of the product.
- **Live Pill.** A Dynamic-Island-style capsule at the top showing what it's doing right now ("Opening Figma...", "Codex editing 3 files"), expandable into detail.
- **Glass Cards.** Results appear as floating frosted cards (weather, a file diff, an email draft) you can dismiss by voice ("close that") or swipe.
- **Agent Dock.** A dock of connected brains/agents with status dots - see who's working, tap to pause.
- **Activity Timeline.** Scrollable history of every action with one-tap / one-phrase undo.
- **Consent Sheets.** Risky actions slide down a sheet and speak the question: "Delete 12 files from Downloads?" - answer yes/no by voice.
- **Command palette fallback.** Cmd-Space-style bar for when you can't talk (meetings, library).

Motion & feel

- Spring physics (Framer Motion / Motion One) - nothing linear.
- backdrop-filter blur + subtle noise texture + 1px inner highlight on every glass surface.
- SF-like typography (Inter or system-ui), generous whitespace, light/dark with vibrancy.
- Sound design: soft earcons for wake, confirm, error - as important as visuals for a voice OS.

7. Form factors & tech stack

Surface	Suggested stack	Notes
Desktop app (start here)	Tauri 2 (Rust core) + React/TS + Motion	Small, fast, real OS access (files, apps, accessibility APIs).
Web app	Same React UI; core runs as a local daemon or cloud	Great for demos; limited device control.
IDE extension	VS Code / JetBrains / Zed extension talking to the daemon	Voice-drive Claude Code / Codex inside the editor.
Core runtime	TypeScript (or Rust) daemon exposing a local WebSocket API	All shells are thin clients to one brain.

One core, many shells: build the agent runtime once as a local service; the desktop app, web app and IDE extensions all just render its state and stream audio to it.

8. Safety - essential for a voice OS

- **Risk tiers:** T0 read-only (just do it) · T1 reversible (do + announce + undo window) · T2 external/irreversible (verbal confirm) · T3 money, credentials, mass delete (tap-to-confirm on screen, never voice-only).
- **Voice ID** so a guest or a YouTube video can't command your machine.

- **Untrusted-content firewall:** anything from web/email/files is labelled and can never raise its own permission level.
- **Undo-by-default:** file ops go to a staging trash; every action is in the audit log.
- **Local-only mode** for sensitive work - routes to on-device models only.

9. Extra ideas to make it stand out

- **Screen awareness:** "What's this error?" - OCR the active window, Jev decides relevance, LLM explains.
- **Routines by voice:** "Every morning, brief me on PRs and calendar" becomes a scheduled workflow.
- **Teach mode:** "Watch me do this once" - records steps, turns them into a reusable skill.
- **Multi-agent hand-offs you can hear:** "Codex is on the refactor, Claude is drafting the release notes."
- **Ambient mode:** the Orb shrinks to a corner and only wakes for high-urgency scores.
- **Multilingual:** English + Twi/Ga support would be a real differentiator in Ghana.
- **Cost meter:** live display of pennies spent per task - Jev makes most of it near zero, which is a great story.

10. Suggested roadmap

Phase	Goal	Done when
0 · Spike (1-2 wks)	Measure Jev latency from your location; build Orb + push-to-talk + STT	"Open Spotify" works end-to-end in < 1 s
1 · MVP (4-6 wks)	Wake word, Jev router, 20 direct skills, one LLM adapter, glass shell	Daily-drivable for simple tasks
2 · Agents	ACP hand-off to Claude Code / Codex, MCP tools, Guardian + undo	"Fix the failing test" by voice
3 · Polish	Addressee detection, barge-in, voice ID, timeline, routines	Feels like Jarvis
4 · Surfaces	Web app + VS Code extension on the same core	Same assistant everywhere

11. Open questions for you

- Primary user: yourself/developers first, or non-technical users?
- Always-listening wake word, or push-to-talk first (simpler, more private)?
- Are you on the Jev early-access list yet? (Access is waitlisted - worth applying now.)
- Name: Nova, Halo, Aura, Echo, Atlas, Sable - or something Ghanaian (e.g. *Nyansa*, Akan for wisdom)?

Sources

TypeSafe AI - Introducing System One Models & Jev (typesafe.ai/blog/introducing-system-one-models-and-jev) · DEV Community - How to Use Jev: a practical guide (dev.to/valyuai) · Check Point - A Decision Model Breaks Like Any Other Language Model (blog.checkpoint.com) · TechCrunch, 18 Sep 2026 · Jev docs: docs.typesafe.ai